Using google I accessed ChatGPT and Gemini AI programs to have them both code this program based on the same prompt. The Prompt I used was:

create a c++ program with these instructions: The program will prioritize emails for a busy company CEO.
• You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
• You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
• You must create objects from scratch. Do not include pre-existing heap modules.
• Test File format:
Command Description
EMAIL <sender category>,
<subject line>,
<date>
The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:
• Boss – read first
• Subordinate – read next
• Peer – read next
• ImportantPerson – read next
• OtherPerson – read last
If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.
EMAIL is followed by space. The rest of the fields are delimited.
Assume <sender category> is one of the five strings listed above.
Assume <subject line> is a string which may contain spaces, but not commas
Assume <date> is in the format: MM-DD-YYYY
NEXT
Next email for the CEO to read. Display the information on the terminal in the following format:
Sender:
Subject:
Date:
READ CEO has read email and dealt with it
COUNT Display current count of unread emails
• The grader will test your code with a different test file. The test file will begin

with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

• Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

• Your program should display this:

```
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.
```

Chat GPT Code:

```
#include <iostream>
#include <string>
#include <sstream>

using namespace std;

// email object
class Email {
public:
```

```
    string sender;
    string subject;
    string date;

    // constructor
    Email(string s, string sub, string d) {
        sender = s;
        subject = sub;
        date = d;
    }
};

// max heap class
class MaxHeap {
private:
    Email** heap;
    int size;
    int capacity;

    // gets the priority of a sender category
    int getSenderPriority(string sender) {
        if (sender == "Boss") {
            return 5;
        }
        else if (sender == "Subordinate") {
            return 4;
        }
        else if (sender == "Peer") {
            return 3;
        }
        else if (sender == "ImportantPerson") {
            return 2;
        }
        else {
            return 1;
        }
    }

    // converts the date into a number for comparison
    int getDateValue(string date) {
        int month = stoi(date.substr(0, 2));
        int day = stoi(date.substr(3, 2));
        int year = stoi(date.substr(6, 4));
```

```
    return year * 10000 + month * 100 + day;
}

// determines whether one email has higher priority
bool higherPriority(Email* first, Email* second) {
    int firstPriority = getSenderPriority(first->sender);
    int secondPriority = getSenderPriority(second->sender);

    // compare sender categories first
    if (firstPriority > secondPriority) {
        return true;
    }

    if (firstPriority < secondPriority) {
        return false;
    }

    // if sender categories are the same compare dates
    return getDateValue(first->date) > getDateValue(second->date);
}

// swaps two emails in the heap
void swapEmails(int first, int second) {
    Email* temp = heap[first];
    heap[first] = heap[second];
    heap[second] = temp;
}

// moves an email upward
void heapifyUp(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;

        if (higherPriority(heap[index], heap[parent])) {
            swapEmails(index, parent);
            index = parent;
        }
        else {
            break;
        }
    }
}

// moves an email downward
```

```
    void heapifyDown(int index) {
        while (true) {
            int left = 2 * index + 1;
            int right = 2 * index + 2;
            int highest = index;

            if (left < size && higherPriority(heap[left], heap[highest])) {
                highest = left;
            }

            if (right < size && higherPriority(heap[right], heap[highest])) {
                highest = right;
            }

            if (highest != index) {
                swapEmails(index, highest);
                index = highest;
            }
            else {
                break;
            }
        }
    }

    // increases the size of the heap array when needed
    void resize() {
        capacity = capacity * 2;

        Email** newHeap = new Email*[capacity];

        for (int i = 0; i < size; i++) {
            newHeap[i] = heap[i];
        }

        delete[] heap;
        heap = newHeap;
    }

public:
    // constructor
    MaxHeap() {
        capacity = 10;
        size = 0;
        heap = new Email*[capacity];
```

```
}

// destructor
~MaxHeap() {
    for (int i = 0; i < size; i++) {
        delete heap[i];
    }

    delete[] heap;
}

// adds an email to the heap
void insert(Email* email) {
    if (size == capacity) {
        resize();
    }

    heap[size] = email;
    heapifyUp(size);
    size++;
}

// returns the highest priority email
Email* peek() {
    if (size == 0) {
        return nullptr;
    }

    return heap[0];
}

// removes the highest priority email
void removeMax() {
    if (size == 0) {
        return;
    }

    delete heap[0];

    heap[0] = heap[size - 1];
    size--;

    if (size > 0) {
        heapifyDown(0);
```

```
        }
    }

    // returns the number of emails
    int getSize() {
        return size;
    }

    // checks if the heap is empty
    bool isEmpty() {
        return size == 0;
    }
};

int main() {
    MaxHeap inbox;

    string line;

    // read the test file one line at a time
    while (getline(cin, line)) {

        // process an EMAIL command
        if (line.substr(0, 5) == "EMAIL") {
            string emailData = line.substr(6);

            string sender;
            string subject;
            string date;

            stringstream ss(emailData);

            // get the three fields separated by commas
            getline(ss, sender, ',');
            getline(ss, subject, ',');
            getline(ss, date);

            // create a new email
            Email* email = new Email(sender, subject, date);

            // add the email to the max heap
            inbox.insert(email);
        }
```

```
        // process a NEXT command
        else if (line == "NEXT") {
            if (!inbox.isEmpty()) {
                Email* nextEmail = inbox.peek();

                cout << "Next email:" << endl;
                cout << "Sender: " << nextEmail->sender << endl;
                cout << "Subject: " << nextEmail->subject << endl;
                cout << "Date: " << nextEmail->date << endl;
            }
        }

        // process a READ command
        else if (line == "READ") {
            if (!inbox.isEmpty()) {
                inbox.removeMax();
            }
        }

        // process a COUNT command
        else if (line == "COUNT") {
            cout << "There are " << inbox.getSize()
                 << " emails to read." << endl;
        }
    }

    return 0;
}
```

Gemini Code:

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <fstream>
#include <algorithm>

// Structure to hold individual Email information
struct Email {
    std::string senderCategory;
    std::string subject;
    std::string date; // Format: MM-DD-YYYY
```

```
    // Helper to get integer rank for priority comparison
    int getCategoryRank() const {
        if (senderCategory == "Boss") return 5;
        if (senderCategory == "Subordinate") return 4;
        if (senderCategory == "Peer") return 3;
        if (senderCategory == "ImportantPerson") return 2;
        if (senderCategory == "OtherPerson") return 1;
        return 0;
    }

    // Converts MM-DD-YYYY into YYYYMMDD integer for chronological comparison
    int getDateComparable() const {
        if (date.length() < 10) return 0;
        int month = std::stoi(date.substr(0, 2));
        int day   = std::stoi(date.substr(3, 2));
        int year  = std::stoi(date.substr(6, 4));
        return year * 10000 + month * 100 + day;
    }

    // Overloaded > operator to determine heap priority
    bool operator>(const Email& other) const {
        int r1 = getCategoryRank();
        int r2 = other.getCategoryRank();

        if (r1 != r2) {
            return r1 > r2;
        }
        // Newer date gets higher priority if categories match
        return getDateComparable() > other.getDateComparable();
    }

    bool operator<(const Email& other) const {
        return !(*this > other || (getCategoryRank() == other.getCategoryRank() &&
                                   getDateComparable() == other.getDateComparable()));
    }
};

// Custom List-based MaxHeap Implementation
class MaxHeap {
private:
    std::vector<Email> heap;

    void heapifyUp(size_t index) {
```

```
        while (index > 0) {
            size_t parent = (index - 1) / 2;
            if (heap[index] > heap[parent]) {
                std::swap(heap[index], heap[parent]);
                index = parent;
            } else {
                break;
            }
        }
    }

    void heapifyDown(size_t index) {
        size_t size = heap.size();
        while (true) {
            size_t largest = index;
            size_t left = 2 * index + 1;
            size_t right = 2 * index + 2;

            if (left < size && heap[left] > heap[largest]) {
                largest = left;
            }
            if (right < size && heap[right] > heap[largest]) {
                largest = right;
            }

            if (largest != index) {
                std::swap(heap[index], heap[largest]);
                index = largest;
            } else {
                break;
            }
        }
    }

public:
    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    bool isEmpty() const {
        return heap.empty();
    }
```

```
    size_t size() const {
        return heap.size();
    }

    Email peek() const {
        if (!isEmpty()) {
            return heap[0];
        }
        return Email();
    }

    void extractMax() {
        if (isEmpty()) return;
        heap[0] = heap.back();
        heap.pop_back();
        if (!isEmpty()) {
            heapifyDown(0);
        }
    }
};

// Helper function to trim whitespace
std::string trim(const std::string& str) {
    size_t first = str.find_first_not_of(" \t\r\n");
    if (first == std::string::npos) return "";
    size_t last = str.find_last_not_of(" \t\r\n");
    return str.substr(first, (last - first + 1));
}

int main(int argc, char* argv[]) {
    // Allows optional command line argument for test file name
    std::string filename = "input.txt";
    if (argc > 1) {
        filename = argv[1];
    }

    std::ifstream inputFile(filename);
    std::istream& input = inputFile.is_open() ? inputFile : std::cin;

    MaxHeap emailQueue;
    std::string line;

    while (std::getline(input, line)) {
        line = trim(line);
```

```
        if (line.empty()) continue;

        if (line.rfind("EMAIL", 0) == 0) {
            // Parse EMAIL record
            std::string data = trim(line.substr(5));
            std::stringstream ss(data);

            std::string sender, subject, date;
            if (std::getline(ss, sender, ',') &&
                std::getline(ss, subject, ',') &&
                std::getline(ss, date)) {

                Email email{trim(sender), trim(subject), trim(date)};
                emailQueue.insert(email);
            }
        }
        else if (line == "COUNT") {
            std::cout << "There are " << emailQueue.size() << " emails to read.\n";
        }
        else if (line == "NEXT") {
            if (!emailQueue.isEmpty()) {
                Email top = emailQueue.peek();
                std::cout << "Next email:\n";
                std::cout << "Sender: " << top.senderCategory << "\n";
                std::cout << "Subject: " << top.subject << "\n";
                std::cout << "Date: " << top.date << "\n";
            }
        }
        else if (line == "READ") {
            if (!emailQueue.isEmpty()) {
                emailQueue.extractMax();
            }
        }
    }

    return 0;
}
```

Code Analysis:

Correctness

Chatgpt's:

The chatgpt's code runs well and does the Max Heap implementation correctly. It uses a dynamically resized array so it is not limited to a certain number of emails. It also correctly compares the sender category first and then the date when two emails have the same sender priority. One issue is that it uses pointers and manual new and delete, which makes memory management more complicated and creates more chances for memory issues if the code is changed later.

Gemini:

Gemini's code also has a good Max Heap implementation and uses a vector to store the emails which makes the memory management simpler than chatgpt's. However it has one important issue with correctness. The peek() function tries to return Email() when the heap is empty but the Email struct does not have a default constructor. This can cause a compile error. Other than that the heap operations and email priority comparisons work similarly to chatgpt's. It also has extra input handling that allows the user to provide a filename or use standard input.

Time Complexity

Chatgpt's:

The code it wrote has pretty good time complexity for its heap operations. Inserting an email takes O(log n) because of heapifyUp and removing an email also takes O(log n) because of heapifyDown. One area that could be improved is the date comparison because getDateValue() repeatedly converts the date string into numbers every time two emails are compared during heap operations.

Gemini:

Similarly to chatgpt's Big O notation shows effective time complexity for the heap operations. Inserting and removing emails are both O(log n) and checking the next email is O(1). The date conversion is also performed repeatedly during comparisons through getDateComparable() so it does not completely solve the issue that chatgpt's has with repeatedly converting dates.

Space Complexity

Chatgpt's:

Chatgpt's code uses a dynamically allocated array for the heap. It starts with space for 10 emails and doubles the capacity whenever the heap becomes full. This means it does not reserve a large fixed amount of space and its memory usage grows as more emails are added.

Gemini:

Gemini's code uses a vector for the heap which automatically grows when more emails are added. This makes the space allocation simpler than chatgpt's because the programmer does

not have to manually create a new array and copy the emails over when the heap gets full. Both codes have O(n) space complexity for storing the emails.

Maintainability

Chatgpt's:

chatgpt's code has a good amount of comments that explain what most of the functions are doing. However it uses raw pointers and manual memory management which can make the code harder to maintain. The code is also separated into different functions which makes the Max Heap easier to follow.

Gemini:

Gemini has comments that help explain the purpose of the different sections and functions. It also uses a vector instead of manually managing an array of pointers which makes the code easier to maintain. The use of overloaded operators for comparing emails also keeps some of the priority logic organized. However the peek() issue with Email() needs to be fixed before the code can be considered fully correct.

Which code I choose:

I think the code that is the best to choose is the gemini code. It has a lot of beneficial changes compared to the chatgpt's code, especially with using a vector and not needing manual memory management for every email. It also has more flexible input options and organizes the email comparison inside the Email structure. However it does need the Email() issue fixed before using it.

Improvements:

So the improvements I will make to the gemini code is to fix the peek() function so it does not try to create an Email() without a default constructor. I would also consider storing the converted date value when the email is created so the date does not have to be converted repeatedly during heap comparisons. Finally I would remove any code that is not needed and add a few more comments to make the code easier to follow.